

Deep L-Layer Neural Networks

Deep vs. Shallow Models

- **Shallow Models:**
 - **Logistic Regression:** Considered a "one-layer" neural network (very shallow).
 - **Single Hidden Layer:** Considered a "two-layer" neural network.
- **Deep Models:** Neural networks with multiple hidden layers.
 - "Depth" is a matter of degree.
 - Deep networks can often learn functions that shallower models are unable to capture.
- **Layer Counting Convention:**
 - **Input Layer:** Indexed as Layer 0. **Not** counted in the total number of layers.
 - **Total Layers (L):** Count of Hidden Layers + Output Layer.
- **Hyperparameter Tuning:** The number of hidden layers is a hyperparameter. It is recommended to try various values (1, 2, or more hidden layers) and evaluate performance on a cross-validation or development set.

Notation for Deep Neural Networks

The following notation is used to describe an L -layer network.

General Parameters

- L : The total number of layers in the network.
- $n^{[l]}$: The number of nodes (units) in layer l .
- $g^{[l]}$: The activation function for layer l (implied).

Layer Indexing

- **Layer 0 (Input):** $n^{[0]} = n_x$ (number of input features).
- **Layer l (Hidden):** $n^{[l]}$ denotes units in the current hidden layer.
- **Layer L (Output):** $n^{[L]}$ denotes the number of units in the final layer.

Activations and Parameters

- $a^{[l]}$: Activations in layer l .
 - Calculated as $a^{[l]} = g(z^{[l]})$.
 - **Input:** $a^{[0]} = x$ (Input features).
 - **Output:** $a^{[L]} = \hat{y}$ (Predicted output).
 - $W^{[l]}$: Weights for computing $z^{[l]}$ in layer l .
 - $b^{[l]}$: Biases for computing $z^{[l]}$ in layer l .
-

Forward Propagation in Deep Networks

1. Single Training Example

To compute the output prediction $\hat{y}$ for a single training example x , forward propagation is performed layer by layer.

- **Input Layer:**
 - The input feature vector is denoted as the activation of layer 0: $a^{[0]} = x$.
- **Layer 1:**
 - $z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$
 - $a^{[1]} = g^{[1]}(z^{[1]})$
- **Layer 2:**
 - $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$
 - $a^{[2]} = g^{[2]}(z^{[2]})$
- **General Recursive Formula:**

For any layer l (where $l = 1, \dots, L$):

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
$$a^{[l]} = g^{[l]}(z^{[l]})$$
- **Output:**
 - The final activation is the predicted output: $a^{[L]} = \hat{y}$.

2. Vectorized Implementation (Entire Training Set)

To process the entire training set efficiently, we stack the examples into columns to form matrices.

- **Notation:**
 - X : The input matrix (training examples stacked in columns).
 - $A^{[0]} = X$.
 - $Z^{[l]}$: Matrix of z values for layer l across all examples.
 - $A^{[l]}$: Matrix of activations for layer l across all examples.

- **Vectorized Equations:**

For layer l :

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

(Note: The bias $b^{[l]}$ is broadcasted across the columns).

3. Implementation Logic

- **Explicit For Loop:**
 - While vectorization usually aims to eliminate loops, an **explicit for loop** is necessary here to iterate through the layers (from $l = 1$ to L).
 - You must compute the activations of layer 1 before layer 2, and so on. There is no way to vectorize across the *depth* (layers) of the network, only across the *data* (training examples).
-

Getting Matrix Dimensions Right

A systematic way to debug and verify deep neural network implementations is to write down and check the dimensions of all matrices and vectors.

1. General Notation & Setup

- L : Total number of layers (excluding input layer).
- $n^{[l]}$: Number of units (nodes) in layer l .
- $n^{[0]} = n_x$: Number of input features.
- m : Number of training examples.

2. Parameter Dimensions (W and b)

The dimensions of the parameters W and b are determined by the number of units in the current layer (l) and the previous layer ($l - 1$). These dimensions remain **constant** regardless of whether you are processing a single example or the entire training set (vectorized).

- **Weights ($W^{[l]}$):**
 - Formula involved: $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 - **Dimension:** $(n^{[l]}, n^{[l-1]})$
- **Bias ($b^{[l]}$):**
 - **Dimension:** $(n^{[l]}, 1)$
- **Backpropagation Gradients:**
 - $dW^{[l]}$ has the same dimension as $W^{[l]}$.
 - $db^{[l]}$ has the same dimension as $b^{[l]}$.

3. Variable Dimensions (Z , A , and X)

The dimensions of activations (A) and linear outputs (Z) change depending on whether the implementation is for a single example or vectorized for m examples.

A. Single Training Example

- **Input (x):** $(n^{[0]}, 1)$
- **Linear Output ($z^{[l]}$):** $(n^{[l]}, 1)$
- **Activation ($a^{[l]}$):** $(n^{[l]}, 1)$

B. Vectorized Implementation (m examples)

Input data is stacked horizontally, where each column represents a single training example.

- **Input (X):** $(n^{[0]}, m)$
- **Linear Output ($Z^{[l]}$):** $(n^{[l]}, m)$
- **Activation ($A^{[l]}$):** $(n^{[l]}, m)$
- **Broadcasting Note:** Even though $b^{[l]}$ is $(n^{[l]}, 1)$, Python/NumPy broadcasting will automatically duplicate it across m columns to match the dimension $(n^{[l]}, m)$ during addition.

Summary Table

Variable	Dimension (Shape)	Note
$W^{[l]}$	$(n^{[l]}, n^{[l-1]})$	Weights connecting layer $l - 1$ to l
$b^{[l]}$	$(n^{[l]}, 1)$	Column vector
$Z^{[l]}, A^{[l]}$	$(n^{[l]}, m)$	m is the batch size (number of examples)
$dW^{[l]}$	$(n^{[l]}, n^{[l-1]})$	Same shape as $W^{[l]}$
$db^{[l]}$	$(n^{[l]}, 1)$	Same shape as $b^{[l]}$

Why Deep Representations?

Why do neural networks with many hidden layers (deep networks) often perform better than those with few layers (shallow networks)? There are two main intuitions: hierarchical feature learning and circuit theory.

1. Intuition: Hierarchical Feature Learning (Simple to Complex)

Deep networks compute functions by building up from simple concepts to complex abstractions. This is often described as a **compositional representation**.

- **Example: Face Recognition**
 - **Early Layers:** Detect simple functions, such as edges (horizontal, vertical, diagonal) in small regions of the image.
 - **Middle Layers:** Compose edges to detect parts of faces (eyes, noses, ears).
 - **Later Layers:** Compose parts to recognize complete faces.
- **Example: Speech Recognition**
 - **Early Layers:** Detect low-level audio waveform features (pitch, ascending/descending tones, white noise).
 - **Middle Layers:** Compose waveform features to detect basic units of sound (**phonemes**).
 - **Later Layers:** Compose phonemes to recognize words, then phrases, and finally sentences.
- **Biological Inspiration:** This hierarchy is loosely inspired by the human brain

(specifically the visual cortex), which is believed to process information in a similar simple-to-complex sequence.

2. Intuition: Circuit Theory (Computational Efficiency)

Mathematical results from circuit theory suggest that certain functions are exponentially more efficient to compute using deep architectures compared to shallow ones.

- **The Problem:** Computing the **parity** (Exclusive OR / XOR) of all input features ($x_1, x_2, \dots, x_n$).
- **Deep Approach (Tree Structure):**
 - You can build an XOR tree where inputs are paired and XORed, and those results are XORed, etc.
 - **Cost:** The depth of the network is on the order of $O(\log n)$. The number of nodes (gates) is small.
- **Shallow Approach (1 Hidden Layer):**
 - If restricted to a single hidden layer, the network must exhaustively enumerate all possible input configurations that result in a specific output (0 or 1).
 - **Cost:** The number of hidden units required is on the order of $O(2^n)$ (exponential).
- **Takeaway:** There are mathematical functions that are "easy" (small) for deep networks to calculate but require exponentially large models if forced into a shallow structure.

3. Terminology and Practical Advice

- **Branding:** The term "Deep Learning" is effectively a rebranding of "Neural Networks with many hidden layers." The catchy branding helped the field gain popularity, alongside the actual performance benefits.
 - **Hyperparameter Tuning:** While deep networks are powerful, one should not blindly maximize depth.
 - Start with simple models (Logistic Regression or 1-2 hidden layers).
 - Treat the **number of hidden layers** as a hyperparameter.
 - Evaluate different depths on cross-validation or development sets to find the optimal architecture for the specific problem.
-

Building Blocks of Deep Neural Networks

To implement a deep neural network efficiently, we can view each layer as a distinct module consisting of a forward function and a corresponding backward function.

1. Modular Architecture per Layer (l)

For any given layer l , the computations can be broken down into two main steps:

A. Forward Propagation Step

- **Input:** Activations from the previous layer, $a^{[l-1]}$.
- **Computation:**
 - $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$
 - $a^{[l]} = g^{[l]}(z^{[l]})$
- **Output:** Activations for the current layer, $a^{[l]}$.
- **Cache:** The value $z^{[l]}$ (and optionally parameters $W^{[l]}, b^{[l]}$) is stored in a **cache** to be used later during backpropagation.

B. Backward Propagation Step

- **Input:**
 - Gradient of the cost with respect to current activations: $da^{[l]}$.
 - **Cache:** Retrieves $z^{[l]}$ (and parameters) stored during the forward step.
- **Computation:** Calculates gradients necessary for updates.
 - Computes $dz^{[l]}$.
 - Computes gradients for parameters: $dW^{[l]}$ and $db^{[l]}$.
- **Output:** Gradient with respect to the *previous* layer's activations: $da^{[l-1]}$.

2. Full Network Pipeline

Combining these blocks creates the full training iteration.

The Forward Pass

1. Initialize with input features $a^{[0]} = x$.
2. Iterate through layers 1 to L :
 - Input $a^{[l-1]}$ into the Forward function.

- Output $a^{[l]}$ and store variables in **Cache**.
3. Final output is $a^{[L]} = \hat{y}$.

The Backward Pass

1. Initialize with the gradient of the loss function with respect to the output.
2. Iterate backwards from layer L to 1:
 - Input $da^{[l]}$ and the corresponding **Cache** into the Backward function.
 - Output $da^{[l-1]}$ (which becomes the input for the next backward step).
 - Output parameter gradients $dW^{[l]}$ and $db^{[l]}$.

3. Implementation Detail: The Cache

- **Conceptual:** The cache primarily stores $z^{[l]}$ because it is required to compute derivatives in the backward step.
- **Practical Implementation:** It is often convenient to store $W^{[l]}$ and $b^{[l]}$ in the cache as well. This simplifies data management by copying parameters to where they are needed for the backward function without using global variables.

4. Parameter Update

After one full pass (Forward + Backward), the parameters are updated using the computed gradients:

$$W^{[l]} = W^{[l]} - \alpha \cdot dW^{[l]}$$

$$b^{[l]} = b^{[l]} - \alpha \cdot db^{[l]}$$

Forward and Backward Propagation Implementation

This section details the specific equations required to implement the Forward and Backward functions for a Deep L -layer Neural Network.

1. Forward Propagation

The goal of the forward pass is to compute the activations for each layer and cache values needed for the backward pass.

- **Input:** $A^{[l-1]}$ (Activations from previous layer).
- **Output:** $A^{[l]}$ (Activations of current layer) and **Cache** ($Z^{[l]}$).

- **Implementation Note:** Storing $W^{[l]}$ and $b^{[l]}$ in the cache is also recommended for ease of programming.

Vectorized Equations (for m examples):

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

- **Initialization:** The first layer receives the input features: $A^{[0]} = X$.

2. Backward Propagation

The goal of the backward pass is to compute the gradients of the cost function with respect to the parameters (W, b) and the activations of the previous layer.

- **Input:** $dA^{[l]}$ (Derivative of cost w.r.t current activation).
- **Output:** $dA^{[l-1]}, dW^{[l]}, db^{[l]}$.

Vectorized Equations:

1. Compute gradient of linear output (dZ):

$$dZ^{[l]} = dA^{[l]} * g'^{[l]}(Z^{[l]})$$

(Note: $*$ denotes element – wise product. g' is the derivative of the activation function).*

2. Compute gradient of weights (dW):

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]T}$$

3. Compute gradient of biases (db):

$$db^{[l]} = \frac{1}{m} \sum_{rows} dZ^{[l]}$$

(Python/NumPy: `np.sum(dZ, axis=1, keepdims=True)`)

4. Compute gradient of previous layer activations (dA_{prev}):

$$dA^{[l-1]} = W^{[l]T} dZ^{[l]}$$

3. Initializing Backpropagation

To start the backward sequence at the final layer L , you must compute the derivative of the cost function with respect to the final prediction $\hat{y}$ (or $A^{[L]}$).

- **For Binary Classification (Log Loss):**

The derivative of the loss function $\mathcal{L}$ with respect to $A^{[L]}$ is:

$$dA^{[L]} = - \left(\frac{Y}{A^{[L]}} - \frac{1-Y}{1-A^{[L]}} \right)$$

- **Vectorized Initialization:**

This formula is applied element-wise across all training examples to create the matrix $dA^{[L]}$, which serves as the input to the backward function for layer L .

4. Summary of Flow

1. **Initialize:** $A^{[0]} = X$.
2. **Forward Prop:** Iterate $l = 1$ to L . Calculate $A^{[l]}$, cache $Z^{[l]}$.
3. **Compute Loss:** Using $A^{[L]}$ and Y .
4. **Initialize Backprop:** Calculate $dA^{[L]}$.
5. **Backward Prop:** Iterate $l = L$ to 1 . Calculate gradients $dW^{[l]}$, $db^{[l]}$ and pass $dA^{[l-1]}$ to the next step.
6. **Update Parameters:** Use gradients to update W and b .

***Note on Complexity:** The code for these algorithms is often surprisingly short. The "magic" of machine learning usually comes from the vast amount of data being processed, rather than thousands of lines of complex code logic.*

Parameters vs. Hyperparameters

Developing deep neural networks requires organizing two distinct types of variables: **Parameters** and **Hyperparameters**.

1. Distinction and Definitions

- **Parameters (W, b):**
 - These are the internal variables (W for weights, b for biases) that the model learns and updates during training through gradient descent.
 - They constitute the final model used for prediction.
- **Hyperparameters:**
 - These are the variables that **control** the "parameters."
 - They are set *before* training begins and determine how the algorithm learns the parameters W and b .
 - Essentially, the value of a hyperparameter determines the final value of the parameters.

2. Examples of Hyperparameters

You must tell the learning algorithm the values for these settings:

- **Learning Rate (α):** Determines the step size during gradient updates.
- **Number of Iterations:** How many times the optimization loop runs.

- **Number of Hidden Layers (L):** The depth of the network.
- **Number of Hidden Units ($n^{[l]}$):** The width of each layer.
- **Activation Functions:** The choice of non-linearity (e.g., ReLU, Tanh, Sigmoid) for hidden layers.
- *Future Examples:* Momentum, mini-batch size, regularization parameters (to be covered in later courses).

3. Deep Learning is an Empirical Process

Because there are so many hyperparameters, it is difficult to know in advance which settings will work best for a specific problem.

- **The Loop: Idea $\rightarrow$ Code $\rightarrow$ Experiment**
 1. **Idea:** Guess a value for a hyperparameter (e.g., $\alpha = 0.01$).
 2. **Code:** Implement the network with that setting.
 3. **Experiment:** Run the training and observe the cost function (J).
 4. **Refine:** Based on the result (e.g., cost diverges or learns too slowly), adjust the value and repeat.
 - **Cross-Domain Intuition:**
 - Intuition gained in one area (e.g., Computer Vision) does not always transfer to another (e.g., NLP or Online Advertising).
 - It is best to remain objective and test a range of values rather than relying solely on past experience from different domains.
 - **Infrastructure Dependency:**
 - Optimal hyperparameters can change over time as infrastructure changes (e.g., CPU vs. GPU upgrades, dataset size increases).
 - **Rule of Thumb:** Even if a model is working, re-evaluate and re-test hyperparameters every few months to ensure they remain optimal.
-

Deep Learning and the Human Brain

While the analogy between Deep Learning and the human brain is popular in media and public discourse, the actual relationship is tenuous and often oversimplified.

1. The Analogy

- **Popular Perception:** Deep learning is often described as "mimicking the brain" because the structural diagrams of neural networks superficially resemble biological neural networks.
- **Biological Neuron:**
 - A cell that receives electrical signals from other neurons ($x_1, x_2, \dots$).
 - Performs a complex thresholding computation.
 - If the threshold is met, it sends an electrical pulse (action potential) down the **axon** to other neurons.
- **Artificial Neuron:**
 - Receives numerical inputs.
 - Performs a linear computation ($z = w^T x + b$) followed by an activation function ($a = \sigma(z)$).
 - Outputs a numerical value.
- **Loose Connection:** The only real similarity is that both receive inputs, perform a computation, and produce an output.

2. Why the Analogy Breaks Down

- **Complexity Mismatch:**
 - A single biological neuron is significantly more complex than a logistic regression unit. Neuroscientists do not yet fully understand the function of a single neuron.
 - Artificial neurons are highly simplified mathematical abstractions.
- **Learning Mechanisms:**
 - **Deep Learning:** Uses **Backpropagation** and **Gradient Descent** to update weights.
 - **The Brain:** It is unknown how the brain actually learns. It is very unclear if the brain uses anything resembling backpropagation or if it operates on a fundamentally different learning principle.

3. Current Perspective

- **Function Approximation:** It is more accurate to view Deep Learning as a method for learning very flexible, complex mathematical functions to map inputs (X) to outputs (Y), rather than as a simulation of the brain.
- **Computer Vision Exception:** The field of Computer Vision has historically taken slightly more inspiration from biological structures (e.g., the visual cortex

hierarchy) than other fields, but the analogy is used less frequently by practitioners today.
